Logger–Blogger: поставка задатка

Елементи развоја софтвера · Тема 02
SOLID и Clean Architecture

1. Опис и циљ задатка

Маја жели да води блог и објављује текстове. Њена сестра Ана је софтверски инжењер и жели да води евиденцију рада пословног софтвера. Обе користе исту апликацију, али у различитим режимима: блогер креира објаве, а логер записе евиденције.

Потребно је реализовати заједнички ток пријаве, креирања и прегледа записа, уз заменљиве сервисе. Циљ је примена SOLID принципа, нарочито Dependency Inversion принципа: апликативни сервиси зависе од интерфејса, а конкретне имплементације добијају кроз конструктор.

Предмет реализације је конзолна апликација у C# на платформи .NET 8, у складу са архивом **Logger–Blogger.zip**. Веб страница из полазне приче представља пословни контекст. Веб интерфејс није део ове поставке.

1.1. Функционални обим

Апликација треба да омогући избор режима Loger или Bloger, пријаву постојећег корисника, додавање записа одговарајуће врсте, преглед тих записа и завршетак сесије. Објављивање враћа експлицитан исход и бележи успех или неуспех у датотеку евиденције активности.

Обезбедити сервис за регистрацију корисника и иницијално креирање демонстрационих налога. Регистрација је део апликативног слоја, док је главни интерактивни ток у приложеном примеру заснован на пријави.

1.2. Границе примера

У овој поставци „ажурирање евиденције” значи додавање нових записа. Измена и брисање постојећег записа нису дефинисани уговором. Тип корисника бира се за текућу сесију, а модел Korisnik не садржи трајну улогу. Тај избор сам по себи не представља систем ауторизације.

2. Структура решења и зависности

Решење организовати у пет извршних и библиотечких пројеката, уз засебан пројекат Tests. Распоред одговорности треба да одговара приложеној архиви.

Табела 1. Пројекти и њихове одговорности

Пројекат	Одговорност и референце
Domain	Модели и енумерације. Нема референци на остале пројекте.
Application	Случајеви употребе, Contracts, интерфејси сервиса, репозиторијума и хеширања, Result типови. Зависи од Domain.
Database	Интерфејси складишта, табеле, in-memory и JSON провајдери, фабрика и репозиторијуми. Зависи од Domain и Application.
Infrastructure	FileLoggerServis и Pbkdf2PasswordHasher. Зависи од Application.
Presentation	ConsoleMenu, IConsoleIO, SystemConsoleIO, AppContainer, seeder, форматер и Program.cs. Повезује потребне слојеве.
Tests	NUnit/Moq тестови модела, сервиса, адаптера и граница слојева.

2.1. Правило зависности

Domain не познаје базу, датотеке, конзолу ни апликативне сервисе. Application дефинише уговоре потребне случајевима употребе, док Database и Infrastructure реализују те уговоре. Тако промена начина чувања или логовања не захтева промену пословних сервиса.

Инстанце конкретних типова састављати на једном месту, у composition root класи AppContainer. Избор базе, репозиторијума, логера и сервиса не распоређивати по пословним класама.

2.2. Примена SOLID принципа

Раздвојити модел, форматирање, чување и интеракцију са корисником. Нови провајдер складишта треба укључити кроз уговор и фабрику. BlogServis и EvidencijaServis морају поштовати исти уговор IZapisiServis, уз експлицитно одбијање погрешне врсте записа.

3. Модел података

Моделе и енумерације сместити у Domain.Models и Domain.Enums. Својства у табели представљају податке који морају бити доступни моделу.

Табела 2. Доменски модели

Тип	Својства и значење
Korisnik	KorisnickoIme: string; LozinkaHash: string; ImePrezime: string. Корисничко име идентификује налог, а лозинка се чува само као изведена хеш вредност.
ZapisNaSajtu	Апстрактна базна класа. Postavljeno: DateTime; Opis: string; Autor: Korisnik. Заједнички подаци свих записа.
Objava	Наслеђује ZapisNaSajtu. Додаје Naslov: string и TipObjave: TipObjave.
Evidencija	Наслеђује ZapisNaSajtu. Додаје TipEvidencije: TipEvidencije.

3.1. Енумерације

```
TipObjave: VLOG = 1, FRIENDS = 2, FAMILY = 3
TipEvidencije: INFO = 1, ERROR = 2, WARNING = 3
```

Конструктори конкретних записа прослеђују датум и време, опис и аутора базном конструктору, а затим постављају сопствена својства. Обезбедити конструкторе и приступ својствима потребан JSON серијализацији.

3.2. Форматирање приказа

Конзолни приказ издвојити у Presentation.Formatting.ZapisFormatter. Метод Format прима ZapisNaSajtu и формира ред табеларног приказа према конкретном типу. За објаву приказати тип, наслов, аутора, опис и време, а за евиденцију тип, аутора, опис и време.

Domain треба да остане независан од ширине конзолних колона. У приложеној архиви форматирање није реализовано редефинисањем ToString() у моделима. За датум се у примеру користи формат **dd.MM.yyyy HH:mm**, а дугачак текст скраћује се само за приказ.

3.3. Улазни уговори

У Application.Contracts дефинисати PrijavaZahtev са корисничким именом и лозинком, и RegistracijaZahtev који додатно садржи име и презиме. Лозинка у улазном захтеву не сме се уписати у модел или лог у чистом облику.

4. Сервиси и резултати операција

4.1. Регистрација и пријава

IAutentifikacijaServis излаже методе Prijava(PrijavaZahtev) и Registracija(RegistracijaZahtev), обе са повратним типом Result<Korisnik>. Имплементацију AutentifikacioniServis сместити у Application.Services.

Регистрација одбија празно корисничко име, лозинку и име и презиме. Име налога и лично име очистити од спољних размака. Не дозволити дупликат корисничког имена, при чему се велика и мала слова не разликују. Грешку читања складишта не тумачити као непостојање корисника.

Пријава проналази налог преко IKorisniciRepozitorijum и проверава лозинку преко IPasswordHasher. Непостојећи налог и погрешна лозинка враћају поруку о неисправним креденцијалима. У Infrastructure реализовати Pbkdf2PasswordHasher са насумичном salt вредношћу и провером хеша. Уговор дефинише методе Hash и Verify. Формат записа је pbkdf2-sha256$iteracije$salt$hash, као у архиви.

4.2. Објављивање и преглед

Интерфејс IZapisiServis има следеће методе:

```
Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis)
Result<IReadOnlyList<ZapisNaSajtu>>
    PregledZapisa()
```

BlogServis прихвата само Objava, а EvidencijaServis само Evidencija. Обе класе припадају Application слоју и добијају IZapisiRepozitorijum и ILoggerServis кроз конструктор. Objavi проверава тип, затим делегира упис репозиторијуму, бележи исход и враћа Result. PregledZapisa враћа резултат читања одговарајућег репозиторијума.

4.3. Result и логовање

Result типови налазе се у Application/Shared, у простору имена Shared. Позивалац најпре проверава IsSuccess или IsFailure. Код грешке користи Error.Code и Error.Message. Failure у приложеној наставној конвенцији добија и конкретну празну вредност одговарајућег типа.

ILoggerServis дефинише bool Loguj(string poruka). FileLoggerServis додаје временски означене поруке у log.txt, враћа true за успешан упис, а false при I/O грешци. Пословна евиденција Evidencija и технички log.txt су различити подаци. У приложеном примеру неуспех логера сам по себи не поништава већ сачуван запис.

5. Складиште и репозиторијуми

5.1. Технички модел складишта

У Database.Models дефинисати TabeleBazaPodataka са листама List<Korisnik> Korisnici, List<Objava> Objave и List<Evidencija> Evidencije. Листе иницијализовати тако да спремно празно складиште може да прихвати први запис.

У Database.Interfaces дефинисати IBazaPodataka са својствима TabeleBazaPodataka Tabele и bool IsReady, и методом bool SacuvajPromene(). IDatabaseProvider излаже Naziv, BazaPodataka и Result<Unit> PokusajKonekciju().

5.2. Провајдери

InMemoryBazaPodataka чува податке само током рада процеса. SacuvajPromene() враћа true без уписа на диск. После поновног покретања нема претходних података.

JsonBazaPodataka учитава podaci.json при креирању, а измене чува у исту датотеку помоћу Newtonsoft.Json. Ако датотека не постоји, почиње са празним табелама. Ако постоји али је неисправна или недоступна, IsReady мора бити false. Такав садржај не сме бити преписан празним табелама.

При чувању прво уписати привремену датотеку, а затим заменити циљну. Неуспех вратити позиваоцу. InMemoryDatabaseProvider и JsonDatabaseProvider повезују конкретно складиште са IDatabaseProvider, а DatabaseProviderFactory бира имплементацију на основу назива in-memory или json.

5.3. Репозиторијуми

У Application.Interfaces.Repositories дефинисати IKorisniciRepozitorijum са методама Dodaj, PronadjiPoKorisnickomImenu и VratiSve, и IZapisiRepozitorijum са методама Dodaj и VratiSve. Повратне вредности су Result типови са одговарајућим моделом или IReadOnlyList колекцијом.

У Database.Repositories реализовати KorisniciRepozitorijum, ObjaveRepozitorijum и EvidencijeRepozitorijum. Репозиторијум записа одбија погрешан конкретни тип. Пре приступа проверити IsReady. При додавању позвати SacuvajPromene(), а ако упис не успе, уклонити управо додату ставку из меморије и вратити грешку. Колекцију за преглед вратити као снимак листе.

6. Интеракција и састављање апликације

Координацију интерактивног тока реализовати у Presentation.Console.ConsoleMenu. Читање и писање менија обављати преко IConsoleIO, чију стварну имплементацију даје SystemConsoleIO. Тако се мени може проверити без интерактивног уноса.

6.1. Ток рада

1. Program.cs креира AppContainer и покреће KorisniciSeeder. Seeder проверава спремност складишта и преко сервиса регистрације додаје демонстрационе налоге ако не постоје. Ако иницијализација не успе, приказати грешку и прекинути покретање.

2. Корисник бира Loger или Bloger. Непознат избор не сме започети сесију. AppContainer.ZapisiServisZa враћа EvidencijaServis или BlogServis за одабрани режим.

3. Мени тражи корисничко име и лозинку, и позива Prijava. При погрешним подацима приказује грешку и дозвољава нов покушај. Празно корисничко име прекида пријаву.

4. Након успешне пријаве доступни су додавање записа, преглед записа и одјава. Аутор новог записа је пријављени корисник. У достављеном примеру NasumicanTekst генерише демонстрациони наслов и опис, а тип записа бира се из одговарајуће енумерације.

5. За додавање позвати Objavi, а за преглед PregledZapisa. Приказати успех или грешку на основу Result-a. Празна листа је успешан преглед без редова. Одјава завршава текући ток менија.

6.2. Composition root

AppContainer повезује IConsoleIO, провајдер базе, три репозиторијума, хешер лозинки, аутентификациони сервис, логер, seeder и оба сервиса записа. ConsoleMenu не треба да креира нову базу или конкретне репозиторијуме приликом сваке операције.

6.3. Покретање приложеног примера

```
dotnet run --project Presentation
dotnet test
```

Команде покренути у корену распакованог решења. У Program.cs је подразумевано AppContainer.Create("in-memory"). За трајно складиште одабрати "json".
Демонстрациони налози су **maja** и **ana**, са лозинком **demo12345**. То су искључиво подаци за наставни пример.

7. Критеријуми прихватања

Задатак је завршен када су одговорности раздвојене, а следећи сценарији имају проверљив исход. NUnit и Moq користити за изоловане тестове сервиса и адаптера, у складу са пројектом Tests из архиве.

Табела 3. Сценарији за проверу решења

Сценарио	Очекивани исход
Исправна пријава	Постојећи налог и исправна лозинка враћају корисника и омогућавају мени.
Неисправна пријава	Погрешна лозинка или непостојећи налог враћају грешку, без успешне сесије.
Неисправна регистрација	Празни обавезни подаци и дупликат имена се одбијају. Грешка базе се прослеђује.
Објава блогера	BlogServis прихвата Objava, чува је преко репозиторијума и покушава упис исхода у лог.
Евиденција логера	EvidencijaServis прихвата Evidencija и користи репозиторијум евиденција.
Погрешан тип записа	Погрешна врста се одбија пре уписа у репозиторијум.
Преглед записа	Враћа се колекција одговарајуће врсте, укључујући исправну празну колекцију.
Неуспешно чување	Враћа се грешка. Додата ставка се повлачи из меморијске листе.
Поновно покретање	JSON режим учитава сачуване податке. In-memory режим почиње изнова.
Оштећен JSON	Складиште није спремно и постојећа датотека остаје сачувана.
Зависности слојева	Domain не референцира друге слојеве. Application не зависи од спољних адаптера.

7.1. Образложење решења

При демонстрацији објаснити како исти IZapisiServis омогућава два режима, где се примењује инјекција зависности и како се складиште замењује без измене апликативних сервиса. Разликовати доменску евиденцију од техничког логовања и грешку операције од грешке приказа.